GCC 128 — INTELIGÊNCIA ARTIFICIAL

Jogo dos Oito

Busca em Amplitude (BFS) × Busca A com Distância de Manhattan.*



PROFESSOR

Ahmed Ali Abdalla Esmin

ALUNO

Tobias Maugus Bueno Cougo

TRABALHO PRÁTICO 05

Objetivo da atividade

Comparar uma busca cega com uma busca informada na resolução do 8-Puzzle.



Busca em Amplitude (BFS)

Método não informado: explora o espaço de estados nível a nível.



Busca A*

Método informado: usa fila de prioridade e a Distância de Manhattan como heurística.



Interface gráfica

Jogar manualmente, resolver com BFS ou A* e navegar pela solução.



Análise de desempenho

Nós expandidos, tempo de execução, redução de nós e speedup.

O PROBLEMA

O Jogo dos Oito (8-Puzzle)

1

Tabuleiro 3x3

Peças de 1 a 8 e um espaço vazio, representado por 0 em uma tupla de 9 posições.

2

Movimentos

Só o espaço vazio se move: Cima, Baixo, Esquerda ou Direita — quando o movimento é válido.

3

Objetivo

Chegar ao estado padrão (1, 2, 3, 8, 0, 4, 7, 6, 5), mostrado ao lado.

4

Solubilidade

Só metade das permutações é alcançável (181.440 estados); a paridade de inversões decide.

Estado objetivo



↑ Cima

↓ Baixo

← Esquerda

→ Direita

FUNDAMENTAÇÃO

BFS × A*

Duas formas de percorrer o mesmo espaço de estados.

Busca em Amplitude (BFS)

- Busca cega: não usa informação do problema
- Fila FIFO: expande por níveis de profundidade
- Conjunto de visitados evita loops
- Ótima aqui, pois cada movimento custa 1

Busca A*

- Busca informada por heurística
- Fila de prioridade (heapq) ordenada por f(n)
- f(n) = g(n) + h(n)
- Ótima se h é admissível (Manhattan é)

Heurística: h(n) = Σ (|linha − linha_objetivo| + |coluna − coluna_objetivo|) sobre todas as peças, exceto o vazio.

IMPLEMENTAÇÃO

O núcleo do A*

Em Python, apenas com a biblioteca padrão (heapq, collections).

Fronteira

Tuplas (f, h, ordem, g, estado) no heapq: menor f primeiro, desempate por menor h.

Fechados e melhor g

Evitam reexpandir estados e descartam caminhos mais caros.

Nó expandido

Estado retirado da fronteira — mesma definição para BFS e A*.

Solução

Caminho reconstruído a partir do pai de cada estado.

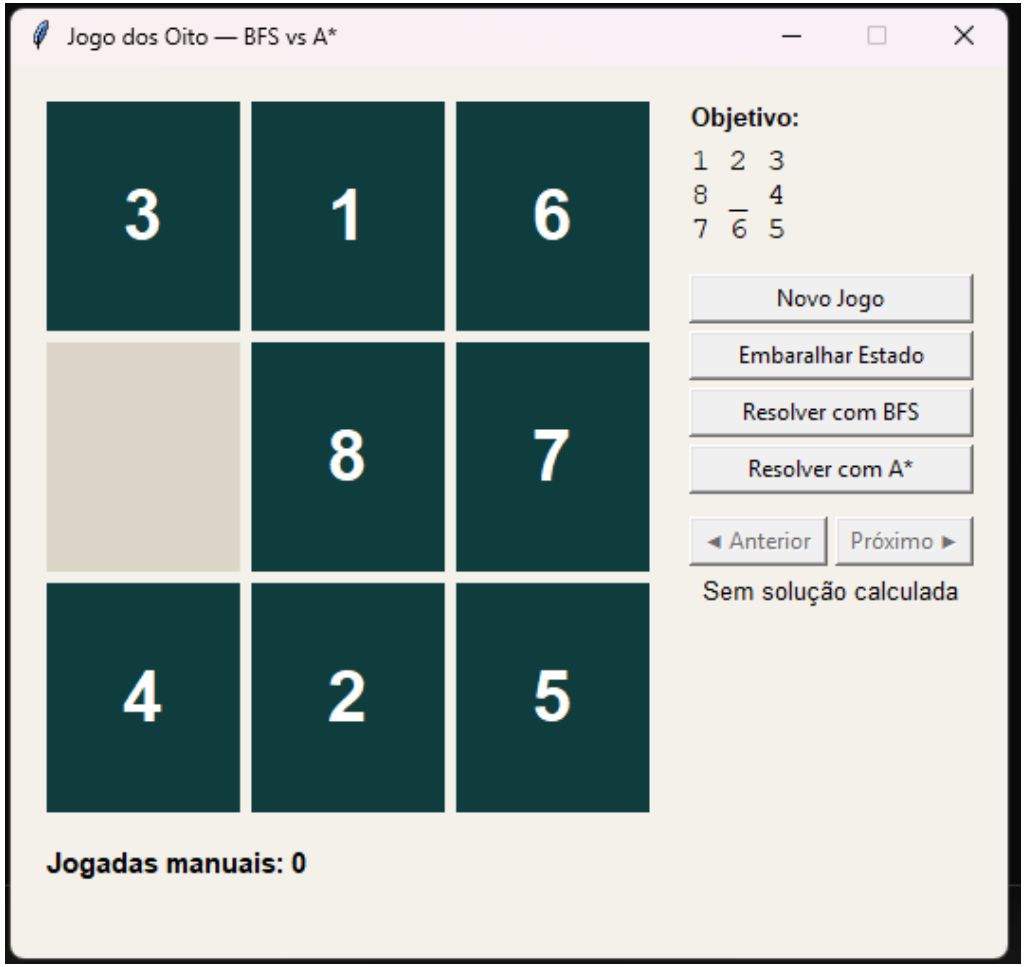
```
fronteira = [(h0, h0, 0, 0, inicio)]
while fronteira:
    _, _, _, g, estado = heappop(fronteira)
    if estado in fechados: continue
    fechados.add(estado); expandidos += 1
    if estado == objetivo:
        return reconstruir(pai, estado)
    for nome, filho in vizinhos(estado):
        novo_g = g + 1
        if filho in fechados or \
           novo_g >= melhor_g.get(filho, inf):
            continue
        melhor_g[filho] = novo_g
        h = manhattan(filho)
        heappush(fronteira,
                 (novo_g + h, h, ordem, novo_g, filho))
```

Trecho simplificado do código (puzzle.py).

INTERFACE GRÁFICA

Jogar, resolver e navegar

Construída com tkinter — print do layout da janela.



Jogada manual

Clique em uma peça adjacente ao vazio.

Novo Jogo / Embaralhar

Gera estados sempre resolvíveis.

Resolver (BFS ou A*)

Mostra passos, nós expandidos e tempo.

Anterior / Próximo

Percorre o histórico da solução.

EXPERIMENTOS

Como comparamos os métodos

Script dedicado (experimentos.py), com resultados reproduzíveis.

50

estados iniciais aleatórios, todos resolvíveis (seed 42)

22,1

passos em média na solução (de 13 a 29)

3

métricas: nós expandidos, tempo (s) e speedup

Redução de nós	(1 − nós A* ÷ nós BFS) × 100
Speedup	tempo do BFS ÷ tempo do A*
Corretude	cada solução é reexecutada; BFS e A* devem ter o mesmo número de passos

RESULTADOS

A* expande 99,4% menos nós

Média dos 50 casos: 94.646 nós no BFS contra 537 no A*.

99,4%

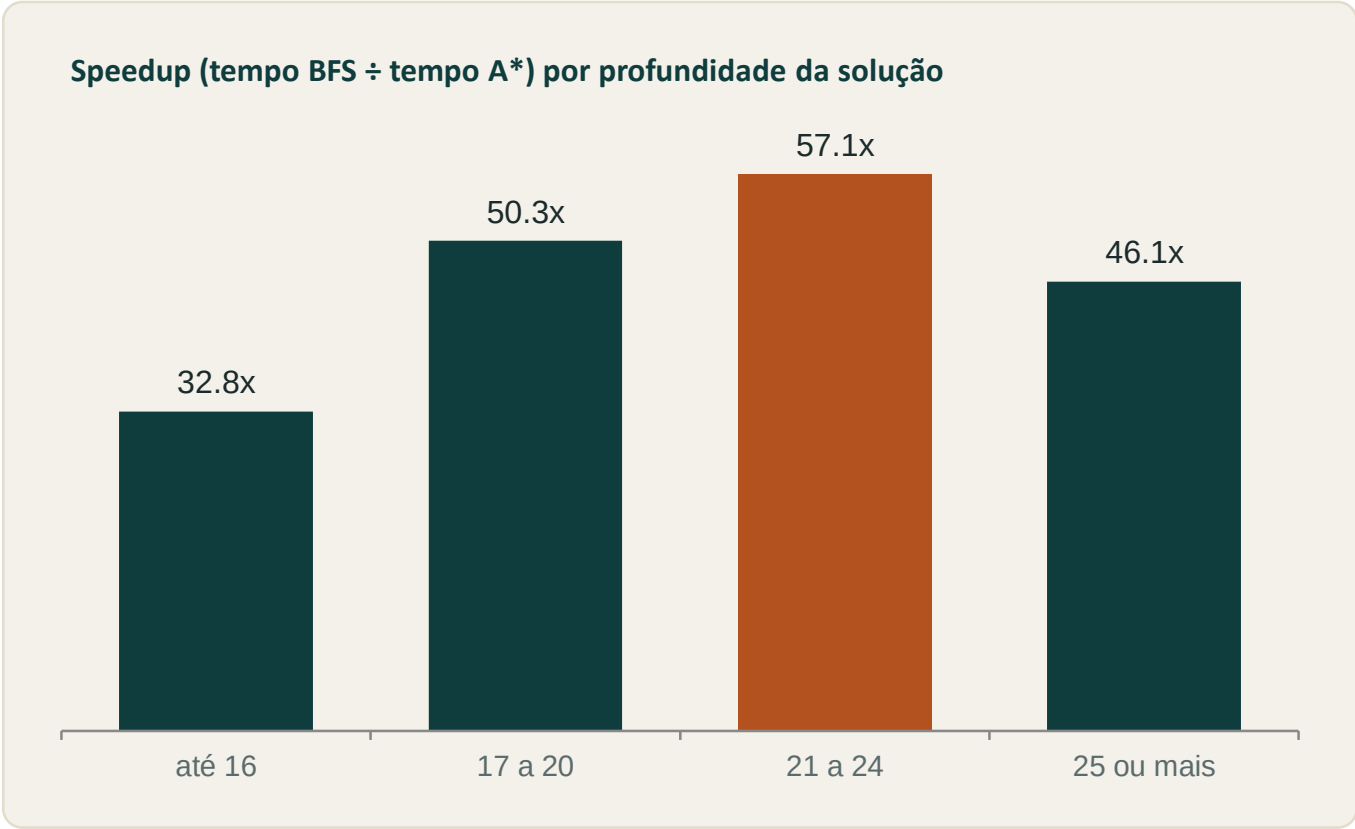
redução de nós

≈ 176×

menos nós expandidos

52,0×

speedup médio no tempo



Tempos medidos em uma execução por caso; dependem da máquina. A redução de nós é determinística.

CONCLUSÃO

O que aprendemos

Heurística boa muda a escala do problema

Com a Distância de Manhattan, o A* expandiu ~0,6% dos nós do BFS e achou a mesma solução ótima.

Ganho de tempo menor que o ganho de nós

Cada nó do A* custa mais; mesmo assim o speedup médio foi de 52x.

BFS explode com a profundidade

Nos casos mais difíceis, chegou a visitar 181.261 dos 181.440 estados alcançáveis.

Obrigado!

Tobias Maugus Bueno Cougo